

chapter 9: Testing Strategy

# Backend testing strategy (Ciné Connect)

This document describes how tests are organized under `apps/backend`, which kinds of tests we run, and why that combination is a reasonable baseline for this Express + Drizzle service.

## Tooling

| Piece                                                  | Role                                                                                               |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Vitest                                                 | Test runner and assertions ( <code>pnpm test</code> , <code>pnpm test:coverage</code> ).           |
| Supertest                                              | HTTP calls against the real Express <code>app</code> instance (integration-style API tests).       |
| <code>.env.test</code> + <code>vitest.config.ts</code> | Loads test env and pins <code>NODE_ENV=test</code> , <code>environment: 'node'</code> .            |
| <code>src/tests/setup.ts</code>                        | Global hooks: DB probe, optional table bootstrap, <b>per-test DB cleanup</b> when DB is available. |

Relevant config:

- Tests are discovered as `src/**/*.test.ts`.
- `fileParallelism: false` reduces cross-test interference when sharing one database.
- Coverage includes `src/**/*.ts` and excludes test files and `index.ts` barrel files.

## Types of tests in this codebase

### 1. Unit tests (fast, isolated)

These import functions or small modules directly and use **mocks** (e.g. mocked `express.Response`) so they do not hit HTTP or the database.

**Purpose:** Lock in contracts for helpers—response shape, status codes, edge cases—at minimal cost.

**Example:** `apiResponse` helpers assert the JSON envelope and status codes:

```
it("returns 200 with envelope by default", () => {
  const payload = { id: 1, name: "x" };
  const out = success(res, payload);
  expect(res.status).toHaveBeenCalledWith(200);
  expect(res.json).toHaveBeenCalledWith({ success: true, data: payload });
});
```

Similar style appears in tests for `sanitizeUserText`, `AppError`, `asyncHandler`, `errorHandler`, etc.

### 2. Integration tests (Supertest + Express app)

These use `request(app)` to exercise middleware, routing, global error handling, and security headers without always needing business-specific DB state.

Purpose: Prove the app behaves correctly as an HTTP server (404 handling, malformed JSON, unauthenticated access, OpenAPI/docs routes).

Example: unknown API route and invalid JSON body:

```
it("returns 404 JSON for unknown /api/v1 route", async () => {
  const res = await request(app).get("/api/v1/this-route-does-not-exist-xyz");
  expect(res.status).toBe(404);
  expect(res.body.success).toBe(false);
  expect(res.body.error).toBe("Route not found");
});

it("returns 400 for malformed JSON on POST", async () => {
  const res = await request(app)
    .post("/api/v1/auth/login")
    .set("Content-Type", "application/json")
    .send("{invalid json}");
  expect(res.status).toBe(400);
  expect(res.body.error).toBe("Invalid JSON");
});
```

### 3. Functional / API tests with database

When PostgreSQL is reachable, `setup.ts` sets `__dbAvailable` and truncates related tables before each test so scenarios start from a clean state. Tests then cover registration, login, cookies, validation, movies, ratings, users, avatars, etc.

Purpose: Validate real persistence and auth flows end-to-end through HTTP + services + Drizzle, which unit tests alone cannot guarantee. Example: user registration and login:

##Example patterns in the repo:

Service-level: duplicate email registration throws a structured `AppError`. REST + DB: forgot-password / reset-password flow with test doubles for outbound email (`__getSentEmailsForTests`). Validation: Zod failures return 400 with `errors[].path` aligned to request shape. Health check adapts to DB availability: 200 + database: 'connected' when the DB is up, 503 when it is not—so CI/local without Docker still gets a defined outcome.

### 4. Schema / migration guard tests

At least one test reads the committed Drizzle SQL and asserts that critical indexes exist (e.g. messages room history). This catches accidental removal of performance-critical DDL during migration edits without requiring a live query planner.

Why this set is “enough” (for this backend)

- Unit tests give fast feedback on pure logic and response formatting; they fail close to the change and are cheap to run everywhere.

- Supertest against the real app ensures wiring is correct: routes, auth middleware, JSON parsing, and consistent error envelopes—not just isolated functions.
- DB-backed tests cover data integrity and cross-layer behavior (registration, tokens, favorites, validation) that mocks would oversimplify.

Determinism: sequential files + beforeEach cleanup reduce flaky shared-state issues on one database.

Operational guard: migration/SQL checks protect indexes and schema intent that affect production behavior under load.

Together, these layers match a common pyramid for a modular monolith API: many small fast tests at the bottom, fewer but higher-value tests through HTTP and the database at the top.

How to run:

- All tests: pnpm test (from apps/backend).
- Coverage: pnpm test:coverage.
- Database: For full DB-backed behavior, run the test database (e.g. Docker) so setup.ts can connect; otherwise some behaviors are skipped or health checks reflect disconnected.

Maintainers: When adding a feature, prefer (1) unit tests for new pure helpers, (2) Supertest for new routes/middleware, and (3) one or two DB-backed flows if the feature writes or reads persisted state. Keep beforeEach cleanup in sync with any new tables that tests populate.